IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 21 February 2026, accepted 24 March 2026, date of publication 2 April 2026, date of current version 9 April 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3680058

TOPICAL REVIEW

Large Language Models for Malware Detection: A Systematic Review, Taxonomy, and Open Challenges

JOYCE CHAMPIE AND KARIM ELISH, (Senior Member, IEEE)
Department of Computer Science, Florida Polytechnic University, Lakeland, FL 33805, USA
Corresponding author: Karim Elish (kelish@floridapoly.edu)

ABSTRACT The rapid evolution of malware targeting mobile devices, IoT/edge platforms, and traditional computing systems has intensified the need for detection techniques capable of reasoning about complex code semantics and behavioral patterns. Large Language Models (LLMs) have recently emerged as a promising direction due to their ability to encode contextual relationships that are difficult to model with traditional approaches. This topical review presents a comprehensive synthesis of LLM-driven malware detection across heterogeneous environments. Based on 26 peer-reviewed studies published between 2020 and 2025, the review introduces a unified taxonomy spanning static and behavioral embeddings, prompt-driven analyses, reasoning-enhanced architectures, and security-oriented fine-tuning strategies. A structured comparative assessment across effectiveness, robustness, computational efficiency, explainability, and privacy, reveals consistent methodological trends: fine-tuned and hybrid transformer architectures deliver the strongest empirical performance, while prompt-based and lightweight models offer practical benefits for resource-constrained platforms. The review also highlights persistent gaps, including limited adversarial-robustness evaluation, insufficient attention to privacy and governance, and a lack of standardized metrics for explainability and energy efficiency. The study concludes by outlining key research challenges and future opportunities, including scalable robustness testing, privacy-preserving LLM adaptation, lightweight architectures for constrained devices, and more rigorous human-centered explainability benchmarks. This review aims to consolidate existing knowledge and guide the development of reliable and practical LLM-driven malware detection systems.

INDEX TERMS Large language model (LLM), malware detection, security, static and dynamic analysis, prompt engineering, fine-tuning, android security.

I. INTRODUCTION

The exponential growth of malware has become one of the most persistent and rapidly evolving threats in the cybersecurity landscape. Traditional malware detection methods, including signature-based, heuristic-based, and conventional machine learning approaches, have shown significant limitations in coping with the speed, sophistication, and diversity of modern malware variants [1], [2], [3], [4]. These systems struggle to detect zero-day attacks, obfuscated codes, and polymorphic threats that dynamically mutate to evade static defenses. As the volume and complexity of malicious software continue to escalate across heterogeneous environments, from Android ecosystems and personal computing devices to Internet of Things (IoT) infrastructures and cloud services, the need for intelligent, adaptive, and context-aware detection techniques has become imperative.

Large Language Models (LLMs), originally developed for Natural Language Processing (NLP) tasks, have recently demonstrated remarkable capabilities that extend beyond linguistic understanding. Their inherent strengths in contextual reasoning, semantic representation, and cross-domain generalization make them particularly suitable for cybersecurity tasks that involve both structured (e.g., API calls, system

The associate editor coordinating the review of this manuscript and approving it for publication was Lorenzo Mucchi.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

logs) and unstructured inputs (e.g., source code, textual descriptions) [5], [6], [7]. Unlike traditional static or dynamic detectors, LLMs can capture complex relationships between code elements and infer potential behavioral intent from static artifacts through self-attention mechanisms and transformer architectures; however, they cannot directly observe runtime execution unless dynamic traces are explicitly provided.

Between 2020 and 2025, the field has witnessed a paradigm shift from static deep learning architectures to transformer-based models specifically adapted for malware analysis [8], [9]. Early studies explored generic pre-trained transformers such as BERT, GPT-2, and GPT-3 [5], [6], while recent efforts have focused on domain-specific adaptations such as VulCoBERT [10], MalBERT [11], and SecureBERT [12]. These models leverage the contextual embeddings of source code and binary features to enhance detection accuracy and interpretability. For instance, AppPoet [8] fine-tuned GPT-based models using opcode sequences and threat intelligence data to generate natural-language rationales for malware classification, demonstrating that LLMs can simultaneously provide both classifications and human-understandable explanations of malicious behavior.

Traditional malware detection techniques are generally categorized into static, dynamic, and hybrid approaches. Static analysis inspects code, metadata, and permissions without execution, offering high scalability but suffering from heavy obfuscation and packing [1], [2]. Dynamic analysis observes behaviors such as API calls, system calls, and network flows in sandboxed environments to capture execution traces invisible to static inspection [13], [14]. However, it is computationally expensive and vulnerable to sandbox evasion [15], [16]. Hybrid approaches aim to combine the strengths of both [3], [17], [18], yet deep contextual reasoning, one of the defining advantages of LLMs, remains underutilized until recent years.

LLM-based malware detection introduces new methodologies that address several persistent limitations of earlier techniques. Recent works explore three major paradigms: (*i*) prompt-engineering-based approaches leveraging zero-shot or few-shot reasoning with models such as GPT-3.5 and GPT-4 [8], [19]; (*ii*) supervised fine-tuning of LLMs on cybersecurity datasets [20], [21]; and (*iii*) reasoning-enhanced architectures that fuse static and dynamic embeddings within transformer-driven frameworks [9], [16]. These approaches enable high detection accuracy (often exceeding 95%) while supporting contextual explanations, multi-view reasoning, and improved robustness to evolving malware [5], [16], [22].

Despite this rapid progress, the literature remains fragmented. Some studies focus exclusively on Android malware [8], [9], [19], [20], while others concentrate on IoT threats [5], [23], [24] or general computing platforms such as Windows and server environments. Inconsistencies across datasets, evaluation metrics, model families, and reporting practices further complicate cross-study comparisons. To date, no comprehensive review has consolidated these efforts into a unified taxonomy coupled with a methodological and quantitative assessment of LLM-based malware detection.

This paper addresses this gap by presenting a *systematic review and taxonomy* of LLM-based malware detection techniques spanning mobile, IoT/edge, and general computing platforms. The review synthesizes findings from twenty-six peer-reviewed studies published between 2020 and 2025, organized along methodological and application-oriented dimensions. Existing survey literature has examined malware detection more broadly, including traditional machine learning, deep learning, and general cybersecurity applications of language models. However, these reviews do not provide a focused synthesis of LLM-based malware detection across mobile, IoT/edge, and general computing platforms, nor do they offer a unified taxonomy and comparative analysis centered on effectiveness, robustness, efficiency, explainability, and privacy.

Specifically, this paper makes the following contributions:

- We systematically review existing research on LLM-based malware detection across mobile, IoT, and general computing platforms, summarizing datasets, architectures, and performance outcomes.
- We propose a four-dimensional taxonomy—*static and behavior embeddings*, *prompt-driven reasoning*, *reasoning-enhanced architectures*, and *security-tuned and fine-tuned LLMs*—that reflects the methodological evolution of the field.
- We conduct a detailed comparison of the surveyed studies across five critical criteria: effectiveness, robustness, computational efficiency, explainability, and privacy considerations.
- We identify key open challenges, including adversarial robustness, continual learning, resource-efficient deployment, data privacy, and explainability—issues that must be addressed to ensure practical, trustworthy, and scalable LLM-based malware detection.

The rest of this paper is organized as follows. Section II reviews the evolution of LLMs and traditional malware detection techniques. Section III outlines our paper selection and evaluation methodology. Section IV presents the proposed taxonomy of LLM-based malware detection approaches. Section V provides a comparative analysis of the surveyed studies, while Section VI discusses open research challenges and future directions. Finally, Section VII concludes the paper by summarizing key insights and identifying opportunities for advancing LLM-based malware detection.

II. BACKGROUND

Malware detection has progressed from traditional static and dynamic analyses to advanced learning-based models. This section first reviews conventional detection approaches and their limitations, then introduces the emergence of Large Language Models (LLMs) as a transformative paradigm that enables contextual and semantic reasoning in security analysis.

A. TRADITIONAL MALWARE DETECTION METHODS

Traditional malware detection systems evolved through successive generations—signature-based, heuristic-based, and machine-learning approaches—each addressing prior weaknesses while introducing new limitations. Understanding this trajectory clarifies how LLMs represent a fundamental departure from conventional paradigms. Malware detection is generally categorized into three principal approaches: static [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], [36], dynamic [15], [37], [38], [39], [40], [41], [42], [43], and hybrid analysis [1], [3], [13], [44], [45], [46].

1) STATIC ANALYSIS APPROACHES

Static analysis inspects application binaries or source code without executing them, focusing on structural and semantic properties of software artifacts. This approach extracts features such as permissions, API calls, and manifest metadata to infer potentially malicious behavior [47], [48]. These static features can be transformed into structured representations suitable for large-scale classification. The efficiency and scalability of static analysis make it attractive for large malware corpora such as [49], [50], [51]. However, its reliance on observable artifacts renders it vulnerable to evasion through obfuscation and packing techniques.

Gibert et al. [52] extended static analysis to general-purpose malware by modeling executables as entropy-based feature streams, while Yuxin et al. [53] demonstrated opcode-sequence modeling to distinguish benign and malicious applications. Despite its computational advantages, static analysis suffers from notable drawbacks: (*i*) limited resilience to code obfuscation and encryption; (*ii*) inability to capture runtime-dependent malicious behaviors such as logic bombs or delayed payloads; and (*iii*) susceptibility to concept drift, as the static patterns of malware evolve over time [9], [15], [54].

2) DYNAMIC ANALYSIS APPROACHES

Dynamic analysis mitigates several static limitations by observing runtime behaviors within controlled environments. It captures API and system call traces, registry modifications, and network interactions that reveal the operational logic of malware [37], [39]. These behavioral traces provide valuable insights into malicious capabilities that static features cannot expose. For instance, Zhou et al. [22] analyzed system call sequences to classify ransomware, reporting high detection performance in both binary and family-level settings, while Sanchez et al. [7] leveraged long-context transformers such as BigBird and Longformer to manage extended system call sequences for IoT malware detection. Despite these advantages, dynamic analysis introduces new challenges, most notably sandbox evasion and high computational overhead. Malware samples can detect virtualized environments and suppress malicious behavior during analysis [15]. Additionally, exhaustive path coverage remains infeasible, leading to partial behavioral visibility [44].

3) HYBRID STATIC AND DYNAMIC ANALYSIS APPROACHES

Hybrid approaches aim to combine the complementary strengths of static and dynamic techniques by integrating code-level features with runtime observations [18], [55]. These frameworks enhance robustness by aligning semantic patterns extracted from source code with behavioral traces captured during execution. Recent studies employ attention-based fusion to improve representation learning across heterogeneous feature spaces [9]. Hybrid methods often achieve higher detection accuracy and better generalization by jointly analyzing, e.g., permissions, API calls, traffic patterns, and execution traces. However, hybrid approaches introduce additional complexity in synchronization, data collection, and feature alignment—factors that motivate the emerging shift toward LLM-driven reasoning architectures capable of unifying these modalities within transformer-based models.

B. EVOLUTION OF LLMS IN SECURITY

Large Language Models (LLMs) have catalyzed a paradigm shift in malware detection by introducing semantic and contextual reasoning capabilities beyond the reach of traditional deep-learning models. Between 2020 and 2025, research in this area evolved from the application of generic pre-trained transformers to the development of domain-specific cybersecurity models tailored to source code and behavioral data. In this review, we use the term ''LLM-based'' broadly to include pretrained transformer-based language models used for malware analysis, including encoder-only, decoder-only, and distilled models such as BERT, GPT, LLaMA, and CANINE.

Early exploratory studies [5], [6] used general-purpose transformer encoders such as BERT and related variants to model logs, binaries, or textual representations of malware, demonstrating that self-attention mechanisms can capture structural and semantic dependencies within software artifacts. Subsequent advances produced cybersecurity-oriented variants such as VulCoBERT [10], MalBERT [11], and SecureBERT [12], which were fine-tuned on security-relevant corpora to improve classification accuracy while maintaining contextual integrity. These specialized LLMs bridge the gap between natural language and code semantics by learning relationships among API calls, permissions, and control-flow or structural representations. For instance, AppPoet [8] demonstrated the potential of multi-view prompt engineering and domain-tuned training by leveraging opcode sequences and threat intelligence texts to generate human-interpretable rationales alongside malware predictions.

More recent efforts highlight the development of architectures that integrate rich static representations with contextual reasoning within transformer-based pipelines. LAMD [9], for example, is trained on a multi-year Android malware dataset and employs context-driven reasoning over static features to improve robustness against evolving families. Similarly,

Li et al. [19] explored cost-efficient adaptation strategies by combining prompt-based zero-shot and few-shot reasoning within models such as ChatGPT, reducing retraining overhead while preserving the performance of underlying detectors. These developments illustrate the gradual transition from purely feature-driven classification toward reasoning-enhanced architectures capable of adapting to evolving malware ecosystems.

Overall, the evolution of LLMs in security signifies a shift from pattern recognition toward contextual understanding and adaptive reasoning. This capability positions LLMs as a foundation for the next generation of malware detection models that integrate static, dynamic, and semantic insights into cohesive, interpretable, and resilient detection pipelines. The following sections build upon this foundation by detailing our methodology for study selection and the taxonomy that categorizes existing LLM-based malware detection research.

III. METHODOLOGY

This study follows a systematic review methodology inspired by the PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) framework [56] to ensure transparency and methodological rigor. The review process consisted of three main stages: (1) identification of relevant literature, (2) screening and eligibility assessment, and (3) inclusion and classification of studies. Figure 1 summarizes the flow of records through each phase.

A. PAPER SELECTION AND CLASSIFICATION

1) SEARCH STRATEGY AND DATA SOURCES

A comprehensive literature search was conducted across major digital libraries, including *IEEE Xplore*, *ACM Digital Library*, *SpringerLink*, *Elsevier ScienceDirect*, and *Google Scholar*. The search covered the publication period from January 2020 to September 2025, corresponding to the rapid emergence of Large Language Models (LLMs) in cybersecurity research. The search query combined three conceptual groups of terms using Boolean operators.

The first group captured model-related keywords: ''Large Language Model,'' ''Transformer,'' ''BERT,'' ''GPT,'' ''LLaMA,'' and ''LLM.'' The second group represented the cybersecurity focus, including ''Malware Detection,'' ''Cybersecurity,'' ''Malicious Code Analysis,'' ''Threat Classification,'' and ''Vulnerability''. The third group targeted computing platforms of interest, including ''Mobile,'' ''Android,'' ''IoT,'' ''Edge,'' and ''Windows''. The final composite query was: (LLM keywords) AND (malware-related keywords) AND (platform keywords). The initial search returned 127 records, which included conference papers, journal articles, and preprints.

2) SCREENING AND ELIGIBILITY CRITERIA

The collected records underwent a multi-stage screening process, consistent with PRISMA guidelines:

(1) Identification: Duplicate studies were removed using automated and manual checks, reducing the total to 108 unique records.

(2) Screening: Titles, abstracts, and keywords were reviewed to exclude studies that were unrelated to malware detection, were not focused on LLMs or transformer architectures, or were purely conceptual without evaluation. This phase excluded 71 papers.

(3) Eligibility: The remaining 37 studies were examined through full-text assessment. Papers were included if they:

- Explicitly applied or analyzed LLMs (e.g., BERT, GPT, LLaMA, CodeBERT, etc.) in the context of malware detection or analysis;
- Reported quantitative results, or qualitative/benchmark-oriented evaluative findings, sufficient to support methodological comparison, even if conventional effectiveness metrics were not always provided;
- Were peer-reviewed and provided clear methodological descriptions;

Exclusion criteria included:

- Studies focusing exclusively on vulnerability detection, phishing, intrusion detection, or anomaly detection without malware analysis;
- Non-peer-reviewed manuscripts;
- Non-English publications.

(4) Inclusion: After full-text evaluation, 26 studies (17 conferences and 9 journals) met all inclusion criteria and were retained for analysis. Figure 1 illustrates the progression through the identification, screening, eligibility, and inclusion phases.

3) CLASSIFICATION OF SELECTED STUDIES

To facilitate structured synthesis, the included studies were classified along two complementary dimensions: *application domain* and *methodological orientation*.

(a) Application Domain:

- **Mobile Malware Detection:** Studies targeting the Android ecosystem, typically leveraging APK files, opcode sequences, manifest features, or permissions.
- **IoT/Edge Malware Detection:** Studies addressing resource-constrained or distributed settings, often emphasizing lightweight models or on-device inference.
- **General Malware Detection:** Studies focusing on traditional computing platforms, such as Windows-based malware analysis leveraging system call traces, memory forensics, or hybrid static–dynamic inputs.

(b) Methodological Orientation:

- **Prompt-driven Reasoning:** Zero-shot or few-shot inference using pre-trained models without further fine-tuning.
- **Fine-tuned LLMs:** Domain-adapted models trained on malware datasets or task-specific objectives.
- **Hybrid Architectures:** Approaches integrating multiple feature modalities or combining LLM reasoning with external tools, graphs, or dynamic traces.

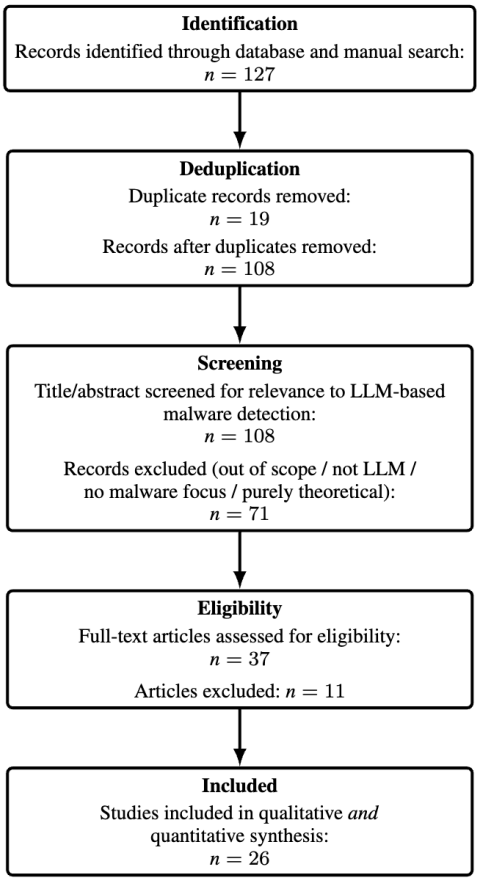


FIGURE 1. PRISMA-style study selection process. The workflow shows the identification, screening, eligibility assessment, and final inclusion of studies in this systematic review.

Each selected paper was coded using a structured extraction form documenting bibliographic metadata, LLM models used, dataset properties, feature modality (static, dynamic, hybrid), evaluation metrics, and methodological orientation. This coding enables the taxonomy introduced in Section IV. Table 1 provides an overview of the included studies, covering publication year, platform/domain, LLM models used, and primary detection methodology. This table establishes the foundation for the taxonomy developed in Section IV and the comparative multi-criteria assessment presented later in Section V.

As with most systematic reviews, this study may be subject to publication bias. The reliance on major English-language digital libraries and the exclusion of non-English publications and non-peer-reviewed venues may omit relevant studies published in other languages or regional outlets. However, these criteria were intentionally adopted to ensure methodological consistency, peer-reviewed quality, and comparability across the surveyed works. In addition, because this is a fast-moving research area, the exclusion of non-peer-reviewed preprints may omit some recent emerging contributions that had not yet been formally published in peer-reviewed venues at the time of the review.

B. EVALUATION CRITERIA

To ensure a consistent and rigorous comparative analysis, we developed a unified evaluation framework derived from prior systematic reviews in machine learning for cybersecurity and adapted to reflect LLM-specific characteristics. The nine criteria are:

1) **Research Objective and Problem Framing:** Defines the specific cybersecurity task addressed (e.g., malware detection, malware family classification).
2) **Model Utilization Strategy:** Identifies how LLMs were employed (e.g., prompt-based reasoning, fine-tuning, or hybrid integration).
3) **Model Architecture:** Captures the type of transformer (e.g., BERT, GPT, CodeBERT, LLaMA) and the structure (encoder-only, decoder-only, encoder–decoder).
4) **Dataset and Feature Source:** Specifies dataset provenance (public vs. private), feature modality (static, dynamic, hybrid), data size, and accessibility.
5) **Evaluation Metrics:** Includes standard measures such as accuracy, F1-score, precision, and recall, as well as specialized interpretability or energy efficiency metrics when reported.
6) **Explainability and Interpretability:** Evaluates whether the study provides interpretable outputs (e.g., natural-language rationales, summaries, attention visualizations, or chain-of-thought reasoning).
7) **Robustness:** Assesses whether the work addresses adversarial robustness, obfuscation resistance, concept drift, or generalization to unseen malware.
8) **Computational Efficiency:** Evaluates whether the study examines computational cost, training overhead, and feasibility for resource-constrained deployment (e.g., IoT or mobile devices).
9) **Privacy:** Assesses whether the study discusses privacy preservation, sensitive data handling, anonymization, or compliance with data-protection constraints during training or deployment.

For each study, these dimensions were extracted using a standardized template. Quantitative metrics (accuracy, F1, precision, recall) and qualitative attributes (robustness, efficiency, explainability, privacy) were recorded and compared across domains.

C. MULTI-DIMENSIONAL COMPARATIVE ASSESSMENT

Following the classification and evaluation phases, each of the 26 included studies was examined through a multi-dimensional comparative lens. The assessment integrates quantitative and qualitative synthesis to capture methodological diversity, performance consistency, and explanatory depth across domains. We reviewed every study against the nine evaluation dimensions introduced in Section III-B. The comparative analysis proceeded across three layers:

1) **Domain-Level Comparison:** Studies were grouped by their target environment—Android/mobile, IoT/edge,

and general computing. This allowed us to analyze how environmental constraints influence the choice of LLM architecture and deployment strategy.

2) **Methodological Comparison:** Within each domain, studies were further divided by their methodological orientation (prompt-driven reasoning, fine-tuning, or hybrid reasoning). Cross-comparison revealed common design patterns such as multi-view prompt engineering, multimodal fusion, and knowledge-graph-assisted reasoning.
3) **Metric-Based Comparison:** Reported performance indicators (e.g., accuracy, F1-score, recall) were compiled from the included studies and compared across methodological classes to highlight general performance trends.

D. RESEARCH QUESTIONS

In alignment with systematic review principles, this study is guided by the following research questions:

- **RQ1:** What are the dominant methodological trends in applying Large Language Models (LLMs) to malware detection across mobile, IoT, and general computing platforms?
- **RQ2:** How do current LLM-based approaches compare in terms of effectiveness, robustness, computational efficiency, explainability, and privacy preservation?
- **RQ3:** What open challenges and research gaps remain to advance reliable and scalable LLM-driven malware detection?

These research questions provide the analytical foundation for the taxonomy and comparative synthesis discussed in subsequent sections.

IV. RQ1: TAXONOMY OF LLM-BASED MALWARE DETECTION APPROACHES

In this section, we propose a comprehensive taxonomy that organizes existing research on LLM-based malware detection into four overarching methodological categories: (1) *Static Code and Behavior Embeddings (SC)*, (2) *Prompt-Driven Reasoning (PD)*, (3) *Reasoning-Enhanced Architectures (RE)*, and (4) *Security-Tuned and Fine-Tuned LLMs (ST)*. Each category represents a stage in the evolution of integrating LLMs into malware analysis pipelines, from data-centric representations toward reasoning-centric and context-aware detection frameworks. These categories are not intended to be strictly mutually exclusive; rather, they represent overlapping methodological dimensions, and individual studies may span multiple categories while still exhibiting a primary methodological orientation.

Figure 2 visualizes the conceptual progression from feature-based to reasoning-driven malware detection paradigms. This taxonomy reflects the methodological spectrum observed in our systematic review and provides a conceptual bridge between traditional detection strategies (discussed in Section II) and modern transformer-based reasoning systems. Table 2 presents the formal definitions and methodological characteristics of each category in our taxonomy.

Given the diversity of computing environments in which LLM-based malware detection is applied, we organize the surveyed studies into three platform domains: mobile systems, IoT/edge environments, and general-purpose platforms. To facilitate comparison, Tables 3, 4, and 5 present a platform-specific comparative evaluation of LLM-based malware detection methods across mobile, IoT/edge, and general computing environments, respectively. Each table summarizes the methodological characteristics of the included studies, including static embeddings (SC), prompt-driven reasoning (PD), reasoning-enhanced architectures (RE), and security-tuned or fine-tuned models (ST), as well as their feature modalities and dataset specifications.

A. STATIC CODE AND BEHAVIOR EMBEDDINGS

Static and behavioral embeddings represent the foundational layer in the taxonomy. These methods translate program artifacts, such as source code, API call sequences, opcode streams, or system logs, into structured textual or numerical embeddings compatible with LLM input formats. This process allows models originally trained for natural language tasks to interpret code semantics and program intent and to bridge the gap between code syntax and human-readable reasoning. It is important to emphasize that, in this category, LLMs operate over static or semi-static artifacts and therefore infer possible behaviors rather than directly observing runtime execution, which remains the role of dynamic analysis.

Unlike traditional static analysis methods that depend on handcrafted features, LLM-based embedding techniques capture semantic and contextual relationships automatically through transformer encoders. By embedding bytecode, API invocations, or system calls into latent vector spaces, models such as BERT, GPT-2, or code-oriented transformers learn representations that generalize across malware families and variants.

Transformer Framework and Code-Level Embeddings. Early studies, including Omar et al. [5] and Demirkiran et al. [6], employed general-purpose transformers such as BERT, GPT-2, and CANINE to encode network traffic, PE features, or API-call sequences into semantic embeddings for classification. In the Android domain, Rahali et al. [11] and Souani et al. [60] used BERT-based encoders over manifest and permission features extracted from AndroZoo [51] applications to perform both binary and family-level malware classification. However, encoder-only models such as BERT and DistilBERT are constrained by input length, typically supporting sequences up to 512 tokens, which can be limiting for long execution traces or highly obfuscated samples [24]. Long-sequence transformers such as BigBird and Longformer, used by Sanchez et al. [7] for IoT malware detection from extended system-call sequences, address this

TABLE 1. Summary of included studies on LLM-based malware detection (2020–2025).

Study	Year	Platform / Domain	LLM / Model Used	Task / Methodology
AppPoet [8]	2025	Mobile (Android)	GPT-4-1106-preview	LLM-assisted multi-view Android malware detection (prompted summaries + DNN classifier)
LLM-MalDetect [20]	2025	Mobile (Android)	Mistral-7B, glm-4-flash	LoRA-fine-tuned static-feature LLM classifier (LLM-MalDetect)
Lee et al. [57]	2025	Mobile (Android)	GPT-4o, WhiteRabbitNeo-33B	Static code segmentation and threat labeling
Li et al. [16]	2025	Mobile (Android)	UniXcoder, ViT	Multimodal fusion of source code and binary images for malware detection
LAMD [9]	2025	Mobile (Android)	GPT-4o-mini, Gemini 1.5 pro	Context-driven detection and 6-way malware classification (LAMD)
LeoDroid [58]	2025	Mobile (Android)	Qwen2.5-0.5B, Qwen2.5-3B, Qwen2.5-7B, LongAlign-7B	Few-shot multi-label Android malware detection (LeoDroid)
He et al. [59]	2025	Mobile (Android)	CodeLLaMA, StarChat, CodeT5, PLBART	Benchmarking code LLMs on malware summarization and maliciousness scoring (Cama)
Li et al. [19]	2025	Mobile (Android)	ChatGPT	Non-decisional ChatGPT explanations for existing Android malware detectors
MalParse [54]	2024	Mobile (Android)	GPT-4o-mini	Semantic analysis, summarization, and categorization of Android malware (MalParse)
Souani et al. [60]	2022	Mobile (Android)	BERT	Binary and family-level Android malware detection from manifest data
MalBERT [11]	2021	Mobile (Android)	BERT	Static-analysis-based Android malware and family classification (MalBERT)
Rondanini et al. [23]	2025	IoT / Edge	DistilBERT, TinyBERT, Llama-3.2-1B, DistilGPT-2, TinyT5	Fine-tuned LLM classification of IoT/edge system logs
Zhang et al. [61]	2025	IoT / Edge	Llama2-7b-chat	LLM-driven malicious traffic detection with log summarization and embedding-based analysis
Omar et al. [5]	2024	IoT	BERT, GPT-2	Fine-tuned LLM classification of IoT malware on ToN-IoT network traffic
Rodanini et al. [24]	2024	IoT / Edge	DistilBERT, TinyBERT	Lightweight LLM malware detection architecture
Sanchez et al. [7]	2024	IoT / Edge	BigBird, Longformer, BERT, DistilBERT, GPT-2, GPTNeo, Mistral	Transfer-learned LLM detection of IoT malware from long system-call sequences
Park et al. [62]	2025	Metaverse / VR	CodeLLaMA, LLaMA-2/3, Qwen2.5, EXAONE-3.5	Token-length optimization for malicious JavaScript detection
Charoenthanakitkul et al. [63]	2025	Windows PE	LLaMA-3 (reporting)	Traditional ML on EMBER with LLM-generated post-hoc malware reports
SRDC [22]	2025	Windows (Ransomware)	GPT-2	LLM-assisted pretraining on dynamic traces for ransomware detection and family classification
Lang et al. [64]	2025	Windows Memory Forensics	GPT-4o, o1, Gemini-2.0-Flash, Grok-3	LLM-augmented malware detection from memory forensic feature datasets
Balan et al. [21]	2025	Windows	LLaMA2-13B, Mistral-7B, Mixtral-8x7B	Fine-tuned API-call-based malware classifier vs. tree-based ML baselines
Kumar et al. [65]	2024	Network + Windows memory (Hybrid)	LLaMA-3-8B/3.1-8B, SecureBERT	Prompt-chaining hybrid detector with cybersecurity knowledge graph
Patsakis et al. [15]	2024	Windows (PowerShell / Scripts)	GPT-4, Gemini-Pro, CodeLLaMA, Mixtral-8x7B	Zero/few-shot deobfuscation and analysis of real-world malware scripts
Altamimi et al. [66]	2024	SMS Spam + Malware Memory Logs	BERT	Fine-tuned and prompt-based spam/malware text classification
Hossain et al. [67]	2024	Java Source Code	Mixtral-8x7B(Dolphin)	Fine-tuned LLM for malicious vs. benign Java code detection
Demirkiran et al. [6]	2022	Windows PE	BERT, CANINE	Ensemble fine-tuned transformer classification

limitation by using sparse attention mechanisms to capture global context over thousands of tokens.

Li et al. [16] introduced a multimodal fusion framework that leverages large pre-trained models (e.g., UniXcoder and ViT-like image encoders) to jointly encode opcode sequences, API semantics, and image-based representations of Android packages. Their fusion model aligns heterogeneous feature spaces via attention-based integration, achieving an F1-score close to 0.98 on Android malware benchmarks while preserving interpretability. Although not a conversational LLM, this work exemplifies the static code and behavior embedding paradigm extended with large pre-trained transformers and multimodal fusion, serving as a bridge toward more reasoning-centric architectures.

Altamimi et al. [66] employed BERT-based encoders (ArabicBERT and AraBERT) to derive semantic embeddings from SMS content and malware-related memory logs, illustrating the use of encoder-style models as powerful feature

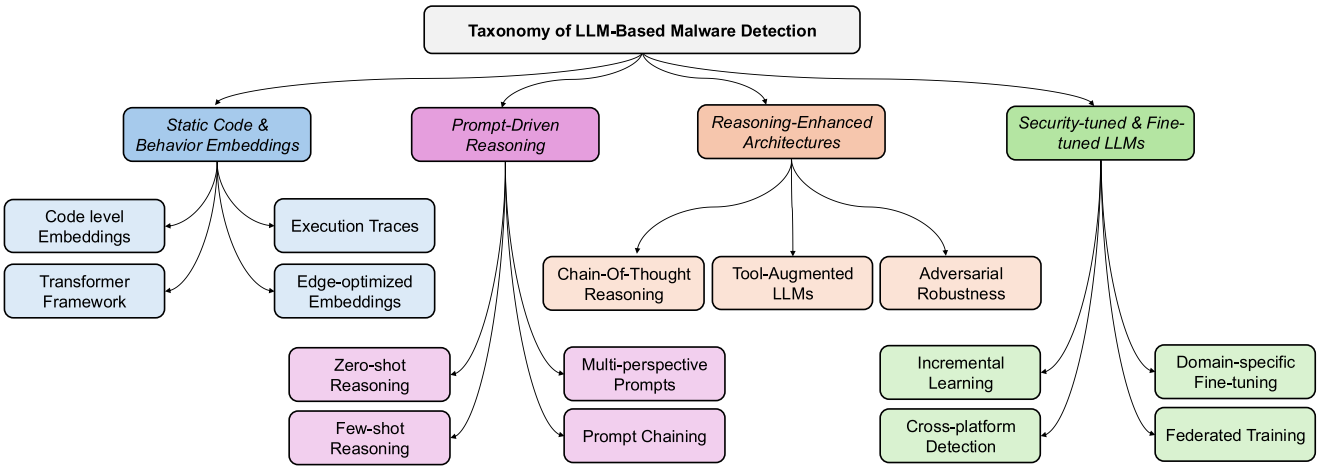


FIGURE 2. Systematic classification of LLM-driven malware detection techniques.

TABLE 2. Definition of taxonomy categories used for classifying LLM-based malware detection methods.

Category	Definition / Methodological Characteristics
Static Code & Behavior Embeddings (SC)	Learned vector representations of static artifacts (APKs, PE files, manifests, permissions, API calls) or semi-static behavior (logs, JavaScript). Uses transformer encoders to capture semantic structure without requiring execution.
Prompt-Driven Reasoning (PD)	LLMs used as reasoning engines via zero-/few-shot prompting, contextual summarization, multi-view prompts, or prompt chaining. No fine-tuning; relies on structured instructions to analyze code, logs, memory, or system artifacts.
Reasoning-Enhanced Architectures (RE)	LLMs combined with external tools, graphs, knowledge bases, or multi-component workflows. Includes multimodal fusion of static + dynamic traces, integration with IDS logs, attack-graph reasoning, and agent-like orchestrations.
Security-Tuned and Fine-Tuned LLMs (ST)	Domain-specific supervised fine-tuning for malware detection, classification, or triage. Includes distilled and lightweight models, training on Android, IoT, or Windows datasets, and comparisons against conventional ML/Deep-learning baselines.

extractors for text-centric malware and spam detection. In the general code domain, Hossain et al. [67] fine-tuned a Mixtral-based model over benign and malicious Java source code, learning rich static embeddings that support high-accuracy malicious code detection.

Tokenization also plays a crucial role in embedding quality. Standard word-level tokenizers often fail to capture code structure, prompting the use of subword-level schemes such as Byte Pair Encoding (BPE) or unigram language models. These tokenizers balance vocabulary compactness with syntactic fidelity, preserving both natural language and programmatic tokens, a vital property when analyzing hybrid artifacts like Android manifests, JavaScript, or mixed opcode–comment corpora.

Edge-Optimized Embeddings and Execution Traces. In IoT and edge scenarios, resource constraints require compact models that maintain accuracy without exceeding hardware limits. Knowledge distillation and pruning have been used to generate lightweight variants such as DistilBERT, TinyBERT, DistilGPT-2, and TinyT5 [23]. These models reduce memory footprint and inference latency while preserving much of the semantic capacity of their teacher models. Rondanini et al. [23] demonstrated that distilled models can effectively encode IIoT log sequences from datasets such as Edge-IIoTset [69], X-IIoTID [70], ToN-IoT [71], and CIC IoT Dataset [72].

Omar et al. [5] investigated BERT and GPT-2 for IoT malware detection over ToN-IoT [71], NSL-KDD, and KDD99 datasets, mapping network flows into token sequences and using LLMs to learn contextual representations of benign versus malicious traffic. Their results show that a fine-tuned encoder (BERT) outperforms an autoregressive model (GPT-2) across multiple metrics. Sanchez et al. [7] further extended static and behavioral embeddings to long system-call sequences via BigBird and Longformer, demonstrating that long-context transformers can capture temporal patterns critical for distinguishing benign from malicious IoT processes.

Transformer-optimized hardware accelerators and parameter-efficient tuning techniques (e.g., LoRA) now enable increasingly capable models to run on constrained platforms. This evolution directly links to **RQ3**, as it addresses scalability and energy-efficiency considerations for real-world LLM-based malware detection.

B. PROMPT-DRIVEN REASONING

Prompt-driven reasoning represents the next methodological layer, leveraging the inherent reasoning abilities of

TABLE 3. Comparative evaluation of LLM-based malware detection methods across mobile platform. SC = Static Code and Behavior Embeddings, PD = Prompt-Driven Reasoning, RE = Reasoning-Enhanced Architectures, ST = Security-Tuned and Fine-Tuned, MF = Malware Family Classification, F = Feature Type (Static / Dynamic / Hybrid), DS = Dataset (M: Malware, B: Benign).

Study	SC	PD	RE	ST	MF	F	DS	DS Type
AppPoet [8]	✓	✓	✓			Static	AndroZoo (M: 12,128; B: 11,189), Drebin (15,036 apps), CICAandMAL (926 apps)	Public
LLM-MalDetect [20]	✓			✓		Static	Drebin (15,036 apps), CICAndMal (926 apps), AndroZoo (4,627 apps, 2,277 apps), RePack (18,073 apps)	Public
Lee et al. [57]		✓				Static	Demonstrated app; no dataset	Private
Li et al. [16]	✓			✓	✓	Static	Data-MD (M: 7,423; B: 7,933), Data-MC (M: 8627; B: 4,042)	Public
LAMD [9]	✓	✓	✓	✓	✓	Static	AndroZoo (13,794 apps)	Public
LeoDroid [58]	✓	✓	✓		✓	Static	anonymousCERT (M: 180 samples), Drebin (M: 5,560 apps, 179 families), VirusShare (exact sample counts not specified)	Public
He et al. [59]	✓	✓			✓	Static	Cama benchmark dataset (M: 118 apps)	Public
Li et al. [19]		✓				Hybrid	Kronodroid (M: 100; B: 100), Drebin and AMD subsets (M: 3,000; B: 3,000)	Public
MalParse [54]		✓				Static	200 APK files (M: 100; B: 100)	Private
Souani et al. [60]	✓			✓	✓	Static	AndroZoo (265K app's manifests)	Public
MalBERT [11]	✓			✓	✓	Static	AndroZoo (M: 10,000; B: 12,000)	Public

TABLE 4. Comparative evaluation of LLM-based malware detection methods across IoT/Edge platform. SC = Static Code and Behavior Embeddings, PD = Prompt-Driven Reasoning, RE = Reasoning-Enhanced Architectures, ST = Security-Tuned and Fine-Tuned, MF = Malware Family Classification, F = Feature Type (Static / Dynamic / Hybrid), DS = Dataset.

Study	SC	PD	RE	ST	MF	F	DS	DS Type
Rondanini et al. [23]	✓		✓	✓	✓	Static	Edge-IIoTset, X-IIoTID, ToN-IoT, CIC IoT Dataset 2023	Public
Zhang et al. [61]	✓	✓	✓			Static	CIC-DDoS2019 + MAWI	Public
Omar et al. [5]	✓			✓		Static	TON-IoT, NSLKDD, KDD99	Public
Rodanini et al. [24]	-	-	-	-	-	-	No dataset; architecture only	-
Sanchez et al. [7]	✓			✓	✓	Dynamic	MalwSpecSys	Public

pre-trained LLMs without necessarily performing full fine-tuning. Instead of retraining on task-specific data, the model is guided through carefully crafted natural-language prompts that encode contextual knowledge of malware behavior, system artifacts, or threat patterns.

This paradigm shift redefines the role of feature engineering: rather than designing manual feature extractors, researchers design linguistic cues that steer the model's inference process. Prompt engineering thus becomes the mechanism for aligning pre-trained knowledge with domain-specific detection tasks.

Zero-Shot and Few-Shot Reasoning. Zero-shot and few-shot prompting approaches exploit the generalization capabilities of large models to detect malicious behavior with minimal labeled data. In zero-shot settings, the model relies entirely on its pre-training, classifying samples based solely on descriptive cues about benign versus malicious behavior [73], [74]. Few-shot prompting supplies a small set of labeled examples within the prompt to improve task-specific alignment.

In the Android domain, Zhao et al. [8] (AppPoet) use GPT-4-class models with multi-view prompts over opcode-based summaries, permissions, and threat-intelligence snippets to perform zero-/few-shot malware detection. Their work shows that prompt-only configurations can approach or surpass traditional deep-learning baselines while also generating natural-language rationales. Li et al. [19] integrate ChatGPT as a decision-support layer on top of existing detectors

TABLE 5. Comparative evaluation of LLM-based malware detection methods across general computing platforms. SC = Static Code and Behavior Embeddings, PD = Prompt-Driven Reasoning, RE = Reasoning-Enhanced Architectures, ST = Security-Tuned and Fine-Tuned, MF = Malware Family Classification, F = Feature Type (Static / Dynamic / Hybrid), DS = Dataset (M: Malware, B: Benign).

Study	SC	PD	RE	ST	MF	F	DS	DS Type
Park et al. [62]	✓	✓				Static	JavaScript files (M: 14,865; B: 407,649)	Private
Charoenthanakitkul et al. [63]	✓					Static	EMBER dataset [68]	Public
SRDC [22]	✓		✓	✓	✓	Dynamic	Dynamic ransomware: 582; B: 942	Private
Lang et al. [64]		✓				Dynamic	Self-developed malware samples Real-world malware from MalwareBazaar	Private Public
Balan et al. [21]	✓			✓		Static	API-sequence dataset (M: 77,911; B: 58,472)	Private
Kumar et al. [65]	✓	✓	✓	✓		Dynamic	CIC-IDS2017, CIC-MalMem-2022	Public
Patsakis et al. [15]		✓				Static	Emotet malware: 2,000 obfuscated PowerShell scripts	Private
Altamimi et al. [66]	✓					Static	SMS Spam Collection (5,574 SMS messages) Malware Memory Analysis dataset	Private
Hossain et al. [67]	✓			✓		Static	32B tokens of benign + malicious Java code	Private
Demirkiran et al. [6]	✓			✓	✓	Static	Catak (7,107 samples), Oliveira (M: 42,797; B: 1,079), VirusSample (M: 9,732), VirusShare (13,849 samples: static API calls)	Public

(Drebin [50] and MaMaDroid [35]) to reassess and justify automated decisions via prompt-based explanations, emphasizing explanation quality over raw detection gains.

Walton et al. [54] propose MalParse, an LLM-based assistant that performs hierarchical semantic summarization and categorization of Android malware using GPT-4o-mini. Their system uses prompts to generate summaries at function, class, and app levels and then relies on these summaries for malware characterization and family-level grouping without model re-training. In the general domain, Patsakis et al. [15] applied GPT-4, Gemini-Pro, CodeLLaMA, and Mixtral to deobfuscate and analyze PowerShell scripts from real-world Emotet malware campaigns [75] via prompt-based interactions, evaluating accuracy and robustness of LLM reasoning under obfuscation.

Multi-Perspective Prompts and Prompt Chaining. More advanced prompt-engineering strategies extend beyond single-step classification. Multi-perspective prompting analyzes malware from several complementary viewpoints (e.g., permissions, API patterns, and control-flow hints) and asks the model to synthesize a final judgment. AppPoet [8] exemplifies this by combining multiple textual views of an Android APK to improve reasoning consistency and resilience to noise.

Prompt chaining decomposes complex analytical tasks into sequential reasoning steps, where each prompt refines or expands on the previous answer. Kumar et al. [65] implement a prompt-chaining approach combined with a cybersecurity knowledge graph, in which LLaMA-3-based models iteratively reason over network flows and memory-dump features to refine malware judgments. In memory forensics, Lang et al. [64] employ GPT-4o, o1, Gemini-2.0-Flash, and Grok-3 through carefully designed prompts to interpret aggregated forensic features, illustrating how prompt-driven reasoning can support the analysis of complex, high-dimensional security data.

C. REASONING-ENHANCED ARCHITECTURES

While prompt-driven reasoning leverages pre-trained knowledge through language alone, reasoning-enhanced architectures explicitly integrate logical or procedural reasoning mechanisms, external tools, or multi-stage workflows around the LLM. These approaches combine transformer-based embeddings with auxiliary components to improve interpretability, robustness, and analytical transparency.

Chain-of-Thought and Context-Driven Reasoning. Chain-of-Thought (CoT) prompting encourages the model to generate intermediate reasoning steps before producing a final answer, revealing the rationale behind each classification. Although not always labeled as "CoT," several surveyed works adopt multi-step or context-driven reasoning patterns. Zhao et al. [8] structure prompts to elicit justifications linked to opcode-derived summaries and threat intelligence, while Kumar et al. [65] decompose decisions over network and memory-dump indicators using chained prompts grounded in a cybersecurity knowledge graph. Qian et al. [9] (LAMD) combine multi-view static features with context-driven reasoning over a nine-year Android dataset from AndroZoo [51], aligning semantic context and family labels to improve robustness against concept drift.

Adversarial Robustness and Tool-Augmented Reasoning. Reasoning-enhanced architectures often integrate external tools, detectors, or knowledge bases into the LLM's workflow. Rondanini et al. [23] propose an architecture in which lightweight LLMs (DistilBERT, TinyBERT, Llama-3.2-1B, DistilGPT-2, TinyT5) are combined with log-parsing pipelines and edge telemetry from multiple IIoT datasets (Edge-IIoTset [69], X-IIoTID [70], ToN-IoT [71], CIC

IoT 2023 [72]). The goal is to support multi-source reasoning under resource constraints. Zhang et al. [61] use LLaMA2-7B in conjunction with traffic summarization and embedding-based analysis to detect malicious flows in CIC-DDoS2019 and MAWI traces, illustrating how context-aware representations and downstream components can be orchestrated for resilience against evolving traffic patterns.

Kumar et al. [65] further demonstrate a hybrid architecture that integrates LLaMA-3 models with a domain knowledge graph and traditional IDS datasets (CIC-IDS2017 [76] and CIC-MalMem-2022 [38]). Their framework supports prompt-chaining over structured security knowledge, yielding a hybrid reasoning pipeline that jointly uses LLM outputs and symbolic relationships. In the ransomware domain, Zhou et al. [22] (SRDC) employ a GPT-2-based pretraining stage over dynamic trace representations followed by task-specific layers, effectively combining sequence modeling with semantic decision boundaries tailored to ransomware families.

D. SECURITY-TUNED AND FINE-TUNED LLMS

At the apex of the taxonomy are security-tuned and fine-tuned LLMs: domain-adapted models explicitly trained for cybersecurity and malware detection tasks. These models combine the contextual understanding of general-purpose LLMs with domain-specific knowledge obtained from large malware corpora, threat intelligence, and code repositories. Fine-tuning can involve full-model retraining, parameter-efficient adaptation (e.g., LoRA, adapters), or incremental updates. Each strategy trades off performance gains against computational and privacy costs.

Domain-Specific Fine-Tuning. Most of the surveyed studies falling under the ST category perform supervised fine-tuning of pre-trained transformers on malware-specific or security-specific datasets within a single platform domain. Feng et al. [20] introduced LLM-MalDetect, which transforms Android APK artifacts into structured string features and fine-tunes LLM-based classifiers over Drebin [50], CICAndMal [77], multiple AndroZoo slices [51], and the RePack [78] dataset. Their results (accuracy up to 98.97%) show that security-tuned LLMs can outperform conventional deep-learning baselines on Android malware detection.

In the IoT domain, Omar et al. [5] fine-tune BERT and GPT-2 on ToN-IoT, NSL-KDD, and KDD99, demonstrating that a tuned encoder-based model can deliver strong performance (up to 98–99% accuracy and F1) on network-based IoT malware detection. Similarly, Sanchez et al. [7] fine-tune BigBird and Longformer on MalwSpecSys [79] system-call traces for IoT malware classification, exploiting long-context attention for extended execution traces.

Rondanini et al. [23] fine-tune lightweight encoders (DistilBERT, TinyBERT, DistilGPT-2, TinyT5) for classifying IIoT logs, balancing performance with edge-resource constraints. Balan et al. [21] train LLaMA2-13B, Mistral-7B, and Mixtral-8 $\times$ 7B over Windows API-sequence datasets and systematically compare their performance against tree-based ML baselines. In the PE malware space, Demirkiran et al. [6] build an ensemble of fine-tuned BERT and CANINE models over VirusShare-derived API-call sequences [80], achieving high AUC values and illustrating the value of transformer fine-tuning at the binary level. Hossain et al. [67] fine-tune a Mixtral-based model on 32B tokens of benign and malicious Java code, positioning LLMs as end-to-end detectors capable of operating directly on source.

Incremental and Continual Learning for Emerging Threats. Because malware evolves rapidly, static fine-tuning can lead to concept drift, where models trained on historical threats struggle with emerging variants. Incremental or continual learning aims to mitigate this gap by periodically updating model parameters as new malware behaviors are observed. SRDC [22] illustrates aspects of this direction by pretraining GPT-2 on dynamic ransomware traces and then adapting to family-level classification and zero-day detection, demonstrating strong results under data-scarce conditions and improved resilience to unseen variants. Although explicit lifelong or streaming-learning protocols are still limited in the current literature, SRDC and LAMD [9] both highlight the importance of temporal coverage (multi-year datasets) and adaptation pipelines for maintaining performance under evolving threats.

Maintaining a balance between adaptability and model stability remains an open challenge, emphasizing the need for continued research into lifelong learning and distribution shift for cybersecurity-oriented LLMs.

Cross-Platform and Multi-Task Detection. Several studies move beyond single-dataset or single-task setups and explore multi-dataset or multi-task configurations within a given platform category, hinting at cross-platform and multi-task capabilities for security-tuned LLMs.

Feng et al. [20] evaluate LLM-MalDetect across multiple Android datasets, while Omar et al. [5] train and test across multiple IoT datasets, effectively treating heterogeneous network datasets as related tasks. Kumar et al. [65] jointly consider network intrusion and malware memory data using LLaMA-3 and SecureBERT within a prompt-chaining and knowledge-graph-assisted framework, and Altamimi et al. [66] combine SMS spam with malware memory logs in a unified text-centric setting.

These works suggest that security-tuned LLMs can simultaneously handle related tasks or data sources, but truly unified cross-platform models that operate seamlessly across mobile, IoT, and general computing ecosystems are still largely unexplored. Designing such models without harming performance on individual domains remains an open research question.

Federated and Privacy-Preserving Training. Federated or privacy-preserving fine-tuning represents an additional, conceptually important sub-dimension of security-tuned LLMs. In principle, federated learning would allow organizations or edge devices to collaboratively train a malware-detection model without sharing raw binaries, logs, or source code, thus reducing privacy and compliance risks.

While several IoT/edge-oriented studies in our survey, such as Rondanini et al. [23] and Zhang et al. [61], emphasize deployment at the edge and discuss resource and privacy constraints, none of the 26 surveyed works implement a fully fledged federated LLM training pipeline. Instead, federated and privacy-preserving fine-tuning currently appears mainly as future directions rather than operational solutions. As LLM-based detectors move closer to real deployments in regulated environments, incorporating federated training, secure aggregation, and on-device adaptation is likely to become a significant future research opportunity.

E. SUMMARY

In summary, this taxonomy demonstrates a clear evolutionary trajectory: from static embeddings that encode malware features, through prompt-driven and reasoning-enhanced architectures that exploit contextual understanding, to security-tuned LLMs that integrate domain expertise and, in some cases, incremental adaptation. Together, these four methodological layers form the conceptual foundation for understanding how LLMs transform malware detection from feature extraction to intelligent, context-aware threat reasoning.

Figure 3 presents a four-set venn diagram showing the distribution of the surveyed studies across the four methodological dimensions defined in our taxonomy. The relationships among categories are more easily interpreted through a visual representation. This visualization highlights the methodological overlap among studies and illustrates how many works draw on multiple paradigms simultaneously.

Beyond methodological categorization, the choice of underlying LLM architectures plays a critical role in shaping the capabilities and performance of malware detection systems. To characterize these architectural trends, we analyze the model families used across all surveyed studies and organize them by platform domain. Figure 4 and Table 6 present the resulting distribution, showing how frequently each LLM family appears in mobile, IoT/edge, and general-platform research. This breakdown reveals clear patterns of adoption. For example, GPT-based and BERT-based models account for a large share of LLM usage in mobile and IoT/edge domains, respectively, whereas general-platform studies exhibit a more heterogeneous mix of GPT, LLaMA, Mistral/Mixtral, and other families.

Answer to RQ1: The systematic synthesis presented in this section identifies four methodological paradigms that define the current landscape of LLM-based malware detection: (1) static and behavioral embeddings, (2) prompt-driven reasoning, (3) reasoning-enhanced architectures, and (4) security-tuned or fine-tuned LLMs. These categories trace a clear trajectory from feature-centric representations toward context-aware, reasoning-driven frameworks. Collectively, they show that the field is transitioning toward integrated and adaptive LLM-based systems capable of multimodal reasoning and, in some cases, continual adaptation to evolving malware ecosystems.

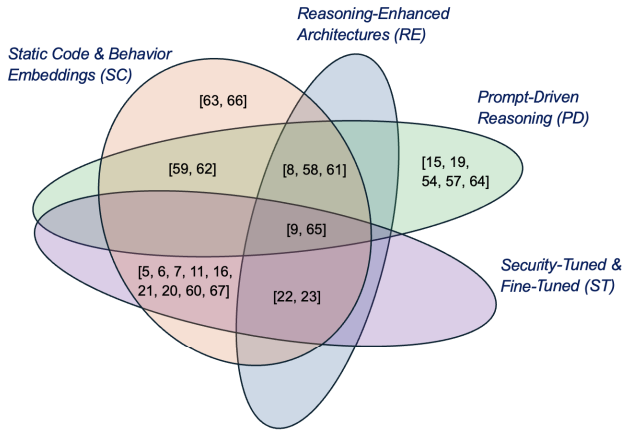


FIGURE 3. Overlap of surveyed LLM-based malware detection studies across the four taxonomy categories. The diagram illustrates how individual studies align with one or more methodological paradigms.

V. RQ2: COMPARATIVE ANALYSIS

This section presents a multi-dimensional comparative analysis of the included studies in this review. Each work was evaluated across five essential dimensions: *effectiveness, robustness, computational efficiency, explainability, and privacy*, which collectively represent the critical criteria for assessing the deployability of LLM-based malware detection systems in real-world environments. These dimensions reflect established machine-learning evaluation practices, while also addressing emerging operational requirements unique to transformer-driven security systems. Table 7 summarizes the comparative findings.

A. EFFECTIVENESS ANALYSIS

Effectiveness, typically measured through accuracy, precision, recall, and F1-score, is the most comprehensively reported dimension. Across all domains, fine-tuned transformer models consistently outperform prompt-based or zero-shot approaches.

For the mobile domain, LLM-MalDetect [20] achieves high effectiveness (Accuracy = 98.97%, F1 97.8%) by fine-tuning on Drebin, CICAndMal2017, and multiple AndroZoo slices. The multimodal Android fusion model of Li et al. [16] similarly reports strong results (Accuracy 97.5%, F1 0.98) by integrating opcode, API, and image-based features. Similarly, prompt-driven systems such as AppPoet [8] achieve lower yet competitive effectiveness (Accuracy 97.1%, F1 97.2%).

In IoT/Edge domains, BERT-based fine-tuning consistently yields high performance. Omar et al. [5] report 98–99% accuracy on ToN-IoT and NSL-KDD, while lightweight models (DistilBERT, TinyBERT, DistilGPT-2, TinyT5) achieve 94–99% accuracy across multiple IIoT datasets in Rondanini et al. [23], demonstrating the viability of compressed LLMs for resource-limited deployments.

In general computing platforms, SRDC-GPT [22] achieves 96% accuracy and F1 = 96% on dynamic ransomware traces, and metaverse-oriented detection achieved 97.3% accuracy in Park et al. [62]. Windows PE analysis using LLaMA2,

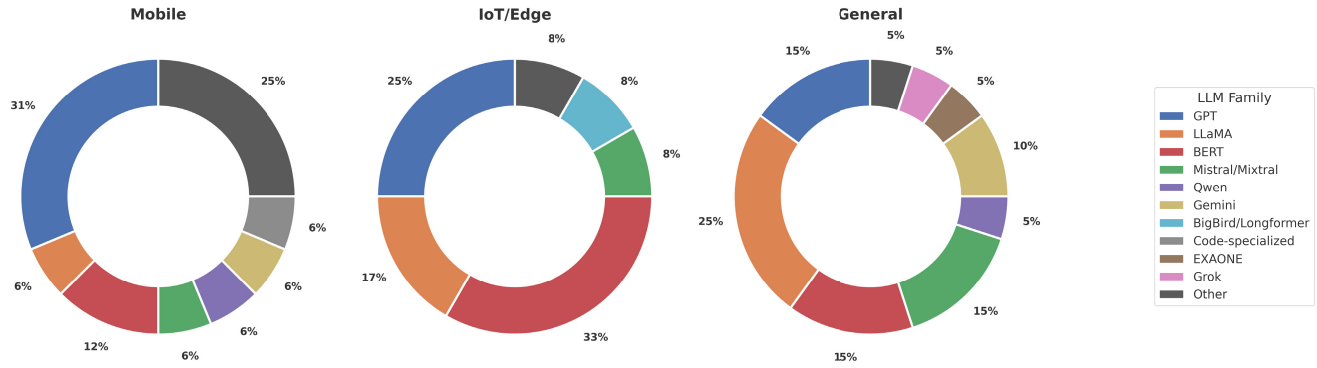


FIGURE 4. Distributions of large language model (LLM) families used across the surveyed malware-detection studies, grouped by platform domain. The visualization illustrates the proportional contribution of different LLM families within each domain.

Mistral, or Mixtral fine-tuning [21] shows effectiveness between 83–91%, depending on model size and API feature set.

A notable limitation across all studies is the scarcity of temporal or family-based validation. Only LAMD [9] performs explicit cross-year testing, revealing that effectiveness drops under dataset shift unless models incorporate incremental learning or continual adaptation.

B. ROBUSTNESS AND SECURITY

Robustness refers to resilience against concept drift, adversarial perturbations, obfuscation, and evasion. Despite strong baseline accuracy, systematic robustness evaluations are rare.

LAMD [9] is one of the few Android studies to conduct cross-year robustness analysis across a 9-year dataset, showing non-trivial temporal degradation without adaptation. In ransomware detection, SRDC-GPT [22] demonstrates improved zero-day performance through semantic pretraining on dynamic traces, highlighting the value of behavior-based contextualization. For long-sequence system-call settings, BigBird/Longformer models [7] retain stable performance under noisy or lengthy traces which suggests inherent robustness to behavioral variability.

However, adversarial robustness remains largely untested. None of the Android APK-focused studies systematically evaluate obfuscation, packing, or adversarial API perturbations. Similarly, IoT studies [5], [23] do not evaluate poisoning, traffic morphing, or prompt-based manipulation. This represents a significant research gap given the active adversarial evolution of malware.

Hybrid reasoning methods such as prompt chaining (e.g., Kumar et al. [65]) offer implicit robustness by enabling multistep validation and self-verification, though explicit adversarial evaluation remains unexplored.

C. COMPUTATIONAL EFFICIENCY

Computational efficiency is a pivotal barrier to deploying LLM-based malware detectors, especially for IoT devices, mobile endpoints, and SIEM/SOAR pipelines.

Large models such as GPT-4o and LLaMA3 family models demonstrate high accuracy but incur substantial memory and inference costs, making them suitable primarily for cloud or offline triage scenarios [16], [64]. In contrast, lightweight LLMs—including DistilBERT, TinyBERT, DistilGPT-2, and TinyT5—used in [23] achieve comparable accuracy while reducing inference latency below 500 ms, supporting near-edge deployment.

For Windows API–sequence tasks, Balan et al. [21] report that 7B-scale models (Mistral-7B, LLaMA2-7B) require substantial GPU memory and training time, whereas Mixtral-8×7B can exceed accuracy but at higher computational cost.

SRDC-GPT [22] demonstrates a balanced design: GPT-2–scale models tailored for ransomware detection achieve high effectiveness while maintaining modest inference latency suitable for endpoint agents.

Measuring computational and energy efficiency should become a standard evaluation metrics, especially for systems targeting edge devices with limited processing and battery capacity. This dimension links directly to **RQ3**, emphasizing the need for sustainable AI security practices.

D. EXPLAINABILITY AND INTERPRETABILITY

Explainability is a distinctive advantage of LLM-based detectors, yet only a subset of the reviewed studies leverage it fully. AppPoet [8] generates human-readable rationales explaining decision pathways based on opcode and threat-intelligence inputs. Li et al. [19] augment Android detectors by prompting ChatGPT to justify the decisions, illustrating the use of LLMs as explainability overlays rather than classifiers. Prompt-chaining approaches [65] produce intermediate reasoning and semantic summaries, offering multi-step interpretability for analysts.

Memory-forensics studies [64] also employ LLM-generated rationalizations to support forensic triage, though explanation quality is not formally evaluated.

However, only a handful of studies attempt to quantify explainability or assess consistency of reasoning, leaving substantial opportunity for structured evaluation frameworks.

E. PRIVACY AND COMPLIANCE

Privacy is the least-addressed dimension across all surveyed studies. Most works rely on public datasets without discussing privacy controls, anonymization procedures,

TABLE 6. Distribution of LLM model families across domains in the surveyed studies (2020–2025). "# Studies" counts the number of papers that employ at least one model from the corresponding family (studies using multiple families are counted in each relevant family).

		Model [Study]		
LLM Family	**# Studies**	**Mobile**	**IoT / Edge**	**General Platform**
GPT	11	GPT-4-1106-preview [8]; GPT-4o [57]; GPT-4o-mini [9], [54]; ChatGPT [19]	DistilGPT-2 [23]; GPT-2 [5], [7]; GPTNeo [7]	o1 [64]; GPT-2 [22]; GPT-4o [64]; GPT-4 [15]
LLaMA	8	CodeLLaMA [59]	Llama-3.2-1B [23]; Llama2-7b-chat [61]	CodeLLaMA [15], [62]; LLaMA-2/3 [62]; LLaMA-3 [63]; LLaMA2-13B [21]; LLaMA-3-8B/3.1-8B [65]
BERT	9	BERT [11], [60]	BERT [5], [7]; DistilBERT [7], [23], [24]; TinyBERT [23], [24]	SecureBERT [65]; BERT [6], [66]
Mistral/Mixtral	5	Mistral-7B [20]	Mistral-7B [7]	Mistral-7B [21]; Mixtral-8x7B [15], [21]; Mixtral-8x7B(Dolphin) [67]
Qwen	2	Qwen2.5-0.5B, Qwen2.5-3B, Qwen2.5-7B [58]	–	Qwen2.5 [62]
Gemini	3	Gemini 1.5 pro [9]	–	Gemini-2.0-Flash [64]; Gemini-Pro [15]
BigBird/Longformer	1	–	BigBird, Longformer [7]	–
Code-specialized	1	CodeT5, PLBART, StarChat [59]	–	–
EXAONE	1	–	–	EXAONE-3.5 [62]
Grok	1	–	–	Grok-3 [64]
Other	6	UniXcoder, ViT [16]; WhiteRabbitNeo-33B [57]; glm-4-flash [20]; LongAlign-7B [58]	TinyT5 [23]	CANINE [6];

or regulatory compliance. No study systematically evaluates federated learning, secure aggregation, encrypted inference, or privacy-by-design methodologies, despite their relevance for regulatory frameworks such as GDPR [81] or NIST AI-RMF [82]. This gap suggests a major future research direction: integrating privacy-preserving fine-tuning and federated training for sensitive malware telemetry.

F. SUMMARY

The comparative findings presented in this section reveal several persistent research gaps that directly inform **RQ3**. The current research landscape is highly imbalanced across evaluation dimensions, as shown in Figure 5. Nearly all reviewed studies report strong *effectiveness*, but far fewer address *efficiency*, *explainability*, or *privacy*. Fine-tuned and hybrid transformer architectures consistently deliver the highest performance; however, interpretability validation and privacy compliance remain underdeveloped. This disparity highlights a pressing need for more holistic evaluation frameworks that extend beyond accuracy to include adversarial resistance, sustainable computation, transparent reasoning, and privacy-preserving model design. Bridging these gaps is essential to ensure that future LLM-based malware detection systems are not only accurate but also trustworthy, auditable, and ethically aligned with real-world deployment requirements.

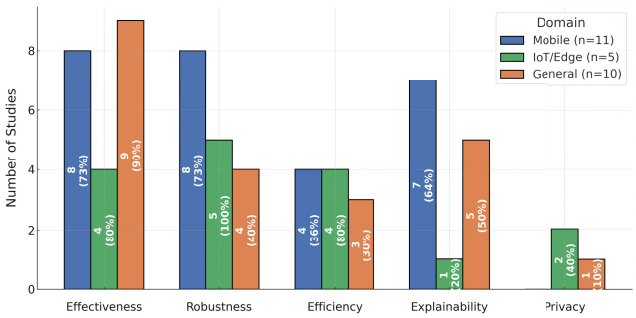


FIGURE 5. Comparative coverage of evaluation criteria across LLM-based malware-detection studies, grouped by platform domain. For each criterion, the three bars represent Mobile (11 studies), IoT/Edge (5 studies), and General Platforms (10 studies). Each bar reports both the number of studies discussing or addressing the criterion (explicitly or partially) and the percentage relative to the total studies within that domain.

Answer to RQ2: Fine-tuned and hybrid LLM architectures provide the highest effectiveness across malware domains, often exceeding 95% accuracy. However, current research lacks systematic robustness testing, energy-efficiency reporting, structured explainability evaluation, and privacy-preserving mechanisms. Addressing these gaps

TABLE 7. Comparative assessment of surveyed LLM-based malware detection studies across five evaluation criteria. Effectiveness values reflect accuracy or equivalent reported performance measures. Symbols: ✓ = criterion explicitly discussed or addressed; ~ = criterion partially discussed or addressed.

Study	Platform/Domain	Effectiveness	Robustness	Efficiency	Explainability	Privacy
AppPoet [8]	Mobile (Android)	Accuracy: 97.1%, F1: 97.2%, Precision: 97%, Recall: 97.3%	✓	✓	✓	
LLM-MalDetect [20]	Mobile (Android)	Accuracy: 90.8–98.9%, F1: 93.7–97.7%, Precision: 94–97.2%, Recall: 92.9–98.3%				
Lee et al. [57]	Mobile (Android)	–	~		✓	
Li et al. [16]	Mobile (Android)	Accuracy: 97.5%, F1: 98%, Precision: 97.7%, Recall: 98.4%	✓	✓	~	
LAMD [9]	Mobile (Android)	F1: 89.85–90.24%	✓	✓	✓	
LeoDroid [58]	Mobile (Android)	MS-ACC: 93.3–97.8%, F1: 64.4–83.5%	✓			
He et al. [59]	Mobile (Android)	–	✓		✓	
Li et al. [19]	Mobile (Android)	–	✓		✓	
MalParse [54]	Mobile (Android)	Accuracy: 77%, Precision: 76%, Recall: 77%	~	✓	✓	
Souani et al. [60]	Mobile (Android)	Accuracy: 97%, F1: 94.9%, Precision: 95.7%, Recall: 94.1%				
MalBERT [11]	Mobile (Android)	Accuracy: 97.61%, F1: 95.47%				
Rondanini et al. [23]	IoT / Edge	Accuracy: 44.1–99.9%, F1: 34.6–99.9%, Precision: 23.7–99.9%, Recall: 49.5–99.9%	✓	✓		~
Zhang et al. [61]	IoT / Edge	F1: 88.9–96.4%	~			
Omar et al. [5]	IoT	Accuracy: 89.7–92.8%, F1: 88.2–94.7%, Precision: 88.3–94.8%, Recall: 88–94.6%	~			
Rondanini et al. [24]	IoT	–	✓	✓		~
Sanchez et al. [7]	IoT	Accuracy: 58.1–86.6%, F1: 59.3–86.8%, Precision: 61.1–87.5%, Recall: 61.8–86.6%	~	~		
Charoenthanakitkul et al. [63]	Windows PE	–			✓	
Kumar et al. [65]	Windows / Hybrid	Accuracy: 91.2% (network packet), 94.3% (memory dump)	~	✓	✓	
SRDC [22]	Windows (Ransomware)	Accuracy: 96%, F1: 96%, Recall: 97%	~	✓	~	
Lang et al. [64]	Windows Memory	Accuracy: 96.9–97.9%, F1: 32–45%, Precision: 13–22%, Recall: 80–100%	~	✓		✓
Balan et al. [21]	Windows	Accuracy: 52.13–91.02%, Recall: 4.16–99.19%		✓		
Park et al. [62]	Metaverse / VR	Accuracy: 87.5–92.8%,, F1: 67.3–92.7%, Precision: 92.5–94.2%, Recall: 81.7–91.4%	~	✓		
Patsakis et al. [15]	Windows (Scripts)	Accuracy: 11.5–69.5% (URL), Accuracy: 29.9–88.7% (domain)	~	✓		✓
Altamimi et al. [66]	Text / Spam	Accuracy: 96.8–100%, F1: 99.9%, Precision: 99.9%, Recall: 99.9% ~				
Hossain et al. [67]	Java Code	–				
Demirkiran et al. [6]	Windows PE	F1: 40.6–80.5%, AUC: 83.2–97.7%		✓		

Note: For certain studies, effectiveness values are reported as ranges, reflecting the variability across all LLM models and datasets evaluated within those studies.

is essential for translating LLM-based malware detection from experimental prototypes into trustworthy, resilient, and compliant real-world systems.

VI. RQ3: OPEN RESEARCH CHALLENGES

Although recent studies demonstrate strong detection effectiveness, the broader goals of robustness, privacy preservation, explainability, and efficiency remain significantly underdeveloped. Based on our systematic review of 26 studies (2020–2025), we identify six overarching research challenges that must be addressed to advance trustworthy and operationally viable LLM-based malware detection systems.

A. ZERO-DAY DETECTION AND CONCEPT DRIFT

A central challenge in malware detection is achieving robust generalization to unseen or *zero-day* threats. Most existing works rely on random or stratified splits, which do not simulate real deployment conditions where malware evolves rapidly. Only a small subset of studies, including LAMD [9], SRDC-GPT [22], and BigBird/Longformer-based IoT detectors [7], explicitly evaluate temporal drift or unseen variants. Results from these works confirm that even high-performing models experience performance degradation when tested on newer samples, highlighting the need for:

- Longitudinal evaluation protocols using temporally disjoint training/testing splits;
- Adaptive learning mechanisms (continual learning, rehearsal buffers, model refreshing);
- Domain-invariant representations that remain stable across evolving malware families.

Future evaluations should also incorporate imbalance-aware and calibration-oriented measures, such as Matthews Correlation Coefficient (MCC), balanced accuracy, and calibration analysis, alongside deployment-oriented metrics including inference latency, FLOPs, and memory footprint.

Given the increasing use of AI-assisted malware generators, zero-day generalization will become an even more critical requirement for next-generation LLM-based defenses.

B. ADVERSARIAL ROBUSTNESS AND SECURITY VALIDATION

Robustness refers to a model's resilience against evasion, adversarial perturbation, and concept drift. Although most LLM-based detectors demonstrate strong baseline performance, our review shows that robustness is typically treated in a limited or implicit manner. Several Android-focused studies (e.g., LAMD [9], AppPoet [8], and related benchmarking work [59]) report cross-family or cross-year evaluations, sensitivity to noisy inputs, or qualitative discussion of obfuscation resistance, which justifies the robustness coverage indicated in Table 7. However, these analyses are usually restricted to dataset splits or scenario-based evaluations rather than comprehensive adversarial testing. In addition to adversarial manipulation, LLM-based systems are inherently susceptible to hallucination and reasoning inaccuracies, where the model may generate plausible but incorrect security explanations or misclassifications, posing risks in high-stakes cybersecurity applications.

Despite the adversarial nature of malware, systematic robustness evaluations remain uncommon. Only a minority of works explicitly consider challenges such as code obfuscation, temporal drift, or distribution shift (e.g., LAMD's cross-year robustness study [9] and SRDC-GPT's zero-day ransomware detection [22]), and almost none of them adopt standardized adversarial benchmarks to probe resistance against API perturbations, behavior morphing, or code-level transformations.

Robustness remains one of the least mature dimensions in LLM-based malware detection. Future work may incorporate:

- Standardized adversarial benchmarks for code, memory forensics, and network data;
- Robustness testing for prompt injection, junk code insertion, and semantic obfuscation;
- Adversarial fine-tuning and consistency regularization;
- LLM red-teaming frameworks tailored to malware analysis pipelines.

C. PRIVACY-AWARE AND FEDERATED LEARNING PARADIGMS

Malware datasets often contain proprietary binaries, system logs, or sensitive user data. However, privacy or compliance considerations are rarely addressed in the surveyed studies. None of the works implements explicit privacy mechanisms. Key gaps include:

- Lack of differential privacy or secure aggregation;
- No federated learning-based malware detectors;
- No analysis of model inversion or data leakage risks;
- Limited discussion of regulatory constraints (e.g., GDPR [81], NIST AI-RMF [82]).

Given increasing collaboration across organizations and global threat intelligence ecosystems, privacy-preserving and decentralized training (e.g., homomorphic encryption, encrypted gradient sharing, federated LLM fine-tuning) will become indispensable.

D. EXPLAINABILITY AND HUMAN-CENTERED EVALUATION BENCHMARKS

Although LLMs are inherently capable of generating natural-language rationales, explainability remains significantly underused and under-evaluated. Only a few studies explicitly emphasize explanation quality: AppPoet [8] produces multi-view explanations for Android malware, Li et al. [19] use ChatGPT to justify malware detectors decisions, Kumar et al. [65] incorporate prompt-chaining and reasoning steps for explainable hybrid detection. Even in these works, explanation fidelity is not measured. Critical missing elements include:

- Standardized explainability benchmarks for malware tasks;
- Human-centered evaluations of explanation usefulness and trustworthiness;
- Tools for visualizing code-level or system-call-level rationales;
- Methods for grounding explanations in verifiable evidence.

To enable adoption in Security Operations Center (SOC) environments, LLM explanations must be reliable, auditable, and aligned with analyst workflows.

E. SUSTAINABLE AND EFFICIENT DEPLOYMENT

LLM-based detection systems face substantial challenges in resource-constrained environments such as mobile devices, IoT sensors, and industrial endpoints. Most high-performing models (GPT-4o, LLaMA3, Mixtral) are impractical for on-device deployment. Studies that address efficiency primarily involve lightweight models:

- DistilBERT, TinyBERT, DistilGPT-2, TinyT5 [23];
- Mixtral/Mistral/LLaMA2-7B fine-tuning [21];
- Low-parameter GPT-2 variants used in SRDC [22].

However, the literature rarely reports inference latency, Floating-Point Operations per Second (FLOPs), energy

consumption, CO_2 impact, and memory footprint. As green AI becomes a priority, future work may investigate:

- Quantization, pruning, distillation, and LoRA for sustainable LLM deployment;
- Neuromorphic accelerators and sparse-attention hardware;
- Optimization for streaming inference (e.g., system-call sequences).

Efficient deployment will determine whether LLM detectors transition from research prototypes to widespread operational use.

F. EVOLVING CYBER THREATS AND LLM MISUSE

The emergence of LLMs introduces a dual-use dilemma. The same models that help detect malware can also generate malicious content: polymorphic code, adaptive evasion routines, automated reconnaissance scripts, and realistic phishing campaigns. Recent research and industry incident reports confirm that attackers are already experimenting with LLM-assisted malware generation. This trend raises the stakes for defensive research and underscores the need for:

- AI-generated malware identification frameworks;
- Monitoring and mitigation of prompt-based evasion strategies;
- Collaborative threat intelligence systems enhanced by LLM summarization;
- Ethical guidelines and governance to prevent misuse of security-oriented LLMs.

Defense strategies must evolve in parallel with offensive AI capabilities.

Answer to RQ3: This analysis identifies six key research challenges: (1) achieving zero-day generalization and mitigating concept drift; (2) developing standardized adversarial robustness frameworks; (3) integrating privacy-preserving and federated learning mechanisms; (4) establishing human-centered explainability benchmarks; (5) enabling sustainable and efficient model deployment; and (6) countering the misuse of generative AI by adversaries. Addressing these challenges is essential to developing next-generation LLM-based malware detectors that are not only accurate but also robust, interpretable, privacy-preserving, and ethically governed.

VII. CONCLUSION

This paper presented a systematic and comprehensive review of recent advances in Large Language Model (LLM)-based malware detection across mobile, IoT/edge, and general computing platforms. Based on 26 peer-reviewed studies published between 2020 and 2025, we synthesized the literature into a four-dimensional taxonomy spanning static and behavioral embeddings, prompt-driven reasoning, reasoning-enhanced architectures, and security-tuned or fine-tuned LLMs. We further conducted a structured multi-criteria comparative assessment covering effectiveness, robustness, computational efficiency, explainability, and privacy, thereby identifying methodological trends as well as persistent gaps.

The findings demonstrate that LLMs have shifted malware detection beyond static, feature-centric classification toward context-aware and reasoning-driven analysis. Fine-tuned and hybrid architectures consistently deliver the strongest empirical performance, while lightweight and prompt-based approaches offer promising efficiency for deployment in constrained settings. However, substantial challenges remain, particularly in adversarial robustness, privacy-preserving model design, standardized explainability evaluation, and energy-aware deployment. These limitations highlight the need for evaluation frameworks that move beyond accuracy to incorporate security, sustainability, and governance considerations required for real-world operational environments.

Advancing the state of LLM-based malware detection will require adaptive, interpretable, and privacy-aware models capable of generalizing to zero-day threats and evolving malware behaviors. Future work should explore continual and federated learning paradigms, rigorous adversarial testing methodologies, energy-efficient model compression, and human-centered evaluation of interpretability. As LLMs continue to mature, they are poised to become integral components of next-generation security operations centers and automated cyber defense pipelines.

Overall, the taxonomy, comparative synthesis, and research agenda presented herein offer a cohesive reference point and a clear trajectory for advancing the next generation of trustworthy, explainable, and resilient LLM-enabled malware detection systems.

REFERENCES

[1] G. Tao, Z. Zheng, Z. Guo, and M. R. Lyu, "MalPat: Mining patterns of malicious and benign Android apps via permission-related APIs," *IEEE Trans. Rel.*, vol. 67, no. 1, pp. 355–369, Mar. 2018.

[2] A. Moser, C. Kruegel, and E. Kirda, "Limits of static analysis for malware detection," in *Proc. 23rd Annu. Comput. Secur. Appl. Conf. (ACSAC)*, Dec. 2007, pp. 421–430.

[3] S. Mahdavifar, D. Alhadidi, and A. A. Ghorbani, "Effective and efficient hybrid Android malware classification using pseudo-label stacked auto-encoder," *J. Netw. Syst. Manage.*, vol. 30, no. 1, p. 22, Jan. 2022.

[4] A. Bensaoud, J. Kalita, and M. Bensaoud, "A survey of malware detection using deep learning," *Mach. Learn. With Appl.*, vol. 16, Jun. 2024, Art. no. 100546.

[5] M. Omar, H. M. Zangana, J. N. Al-Karaki, and D. Mohammed, "Harnessing LLMs for IoT malware detection: A comparative analysis of BERT and GPT-2," in *Proc. 8th Int. Symp. Multidisciplinary Stud. Innov. Technol. (ISMSIT)*, Nov. 2024, pp. 1–6.

[6] F. Demirkıran, A. Çayır, U. Ünal, and H. Dağ, "An ensemble of pre-trained transformer models for imbalanced multiclass malware classification," *Comput. Secur.*, vol. 121, Oct. 2022, Art. no. 102846.

[7] P. M. S. Sánchez, A. H. Celdrán, G. Bovet, and G. M. Pérez, "Transfer learning in pre-trained large language models for malware detection based on system calls," in *Proc. MILCOM IEEE Mil. Commun. Conf. (MILCOM)*, Oct. 2024, pp. 853–858.

[8] W. Zhao, J. Wu, and Z. Meng, "AppPoet: Large language model based Android malware detection via multi-view prompt engineering," *Expert Syst. Appl.*, vol. 262, Mar. 2025, Art. no. 125546.

[9] X. Qian, X. Zheng, Y. He, S. Yang, and L. Cavallaro, "LAMD: Context-driven Android malware detection and classification with LLMs," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2025, pp. 126–136.

[10] Y. Xia, H. Shao, and X. Deng, "VulCoBERT: A CodeBERT-based system for source code vulnerability detection," in *Proc. Int. Conf. Generative Artif. Intell. Inf. Secur.* New York, NY, USA: Association for Computing Machinery, May 2024, pp. 249–252.

[11] A. Rahali and M. A. Akhloufi, "MalBERT: Malware detection using bidirectional encoder representations from transformers," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Oct. 2021, pp. 3226–3231.

[12] E. Aghaei, X. Niu, W. Shadid, and E. Al-Shaer, "SecureBERT: A domain-specific language model for cybersecurity," in *Proc. Int. Conf. Secur. Privacy Commun. Syst.*, 2022, pp. 39–56.

[13] R. Surendran, T. Thomas, and S. Emmanuel, "On existence of common malicious system call codes in Android malware families," *IEEE Trans. Rel.*, vol. 70, no. 1, pp. 248–260, Mar. 2021.

[14] P. Vinod, A. Zemmari, and M. Conti, "A machine learning based approach to detect malicious Android apps using discriminant system calls," *Future Gener. Comput. Syst.*, vol. 94, pp. 333–350, May 2019.

[15] C. Patsakis, F. Casino, and N. Lykousas, "Assessing LLMs in malicious code deobfuscation of real-world malware campaigns," *Expert Syst. Appl.*, vol. 256, Dec. 2024, Art. no. 124912.

[16] X. Li, L. Liu, Y. Liu, Y. Zhao, P. Zhang, and H. Liu, "Multimodal fusion for Android malware detection based on large pre-trained models," *IEEE Trans. Softw. Eng.*, vol. 51, no. 5, pp. 1569–1590, May 2025.

[17] M. Nisa, J. H. Shah, S. Kanwal, M. Raza, M. A. Khan, R. Damaševičius, and T. Blažauskas, "Hybrid malware classification method using segmentation-based fractal texture analysis and deep convolution neural network features," *Appl. Sci.*, vol. 10, no. 14, p. 4966, Jul. 2020.

[18] N. Kumar, S. Mukhopadhyay, M. Gupta, A. Handa, and S. K. Shukla, "Malware classification using early stage behavioral analysis," in *Proc. 14th Asia Joint Conf. Inf. Secur. (AsiaJCIS)*, Aug. 2019, pp. 16–23.

[19] Y. Li, S. Fang, T. Zhang, and H. Cai, "Enhancing Android malware detection: The influence of ChatGPT on decision-centric task," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 8, pp. 1–30, Nov. 2025.

[20] R. Feng, H. Chen, S. Wang, M. M. Karim, and Q. Jiang, "LLM-MalDetect: A large language model-based method for Android malware detection," *IEEE Access*, vol. 13, pp. 81347–81364, 2025.

[21] G. Balan, C.-A. Simion, and D. T. Gavrilut, "Fine tuning LLMs vs non-generative machine learning models: A comparative study of malware detection," in *Proc. 17th Int. Conf. Agents Artif. Intell. (ICAART).*, 2025, pp. 715–725.

[22] C. Zhou, Y. Liu, W. Meng, S. Tao, W. Tian, F. Yao, X. Li, T. Han, B. Chen, and H. Yang, "SRDC: Semantics-based ransomware detection and classification with LLM-assisted pre-training," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 27, pp. 28566–28574.

[23] C. Rondanini, B. Carminati, E. Ferrari, A. Kundu, and A. Gaudiano, "Malware detection at the edge with lightweight LLMs: A performance evaluation," *ACM Trans. Internet Technol.*, vol. 26, no. 1, pp. 1–24, Feb. 2026.

[24] C. Rondanini, B. Carminati, E. Ferrari, A. Kundu, and A. Jajoo, "Large language models to enhance malware detection in edge computing," in *Proc. IEEE 6th Int. Conf. Trust, Privacy Secur. Intell. Syst., Appl. (TPS-ISA)*, Oct. 2024, pp. 1–10.

[25] K. O. Elish, X. Shu, D. Yao, B. G. Ryder, and X. Jiang, "Profiling user-trigger dependence for Android malware detection," *Comput. Secur.*, vol. 49, pp. 255–273, Mar. 2015.

[26] K. O. Elish, M. O. Elish, and H. M. J. Almohri, "Lightweight, effective detection and characterization of mobile malware families," *IEEE Trans. Comput.*, vol. 71, no. 11, pp. 2982–2995, Nov. 2022.

[27] J. Garcia, M. Hammad, and S. Malek, "Lightweight, obfuscation-resilient detection and family identification of Android malware," *ACM Trans. Softw. Eng. Methodol.*, vol. 26, no. 3, pp. 1–29, Jul. 2017.

[28] Y. Zhou, Z. Wang, W. Zhou, and X. Jiang, "Hey, you, get off of my market: Detecting malicious apps in official and alternative Android markets," in *Proc. 19th Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2012.

[29] T. Gao, W. Peng, D. Sisodia, T. K. Saha, F. Li, and M. Al Hasan, "Android malware detection via graphlet sampling," *IEEE Trans. Mobile Comput.*, vol. 18, no. 12, pp. 2754–2767, Dec. 2019.

[30] B. Wolfe, K. O. Elish, and D. Yao, "Comprehensive behavior profiling for proactive Android malware detection," in *Proc. Inf. Secur. Conf. (ISC)*, 2014, pp. 328–344.

[31] D.-J. Wu, C.-H. Mao, T.-E. Wei, H.-M. Lee, and K.-P. Wu, "DroidMat: Android malware detection through manifest and API calls tracing," in *Proc. 7th Asia Joint Conf. Inf. Secur.*, Aug. 2012, pp. 62–69.

[32] K. O. Elish, H. Cai, D. Barton, D. Yao, and B. G. Ryder, "Identifying mobile inter-app communication risks," *IEEE Trans. Mobile Comput.*, vol. 19, no. 1, pp. 90–102, Jan. 2020.

[33] W. Blanc, L. G. Hashem, K. O. Elish, and M. J. H. Almohri, "Identifying Android malware families using Android-oriented metrics," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2019, pp. 4708–4713.

[34] B. Wolfe, K. Elish, and D. Yao, "High precision screening for Android malware with dimensionality reduction," in *Proc. 13th Int. Conf. Mach. Learn. Appl.*, Dec. 2014, pp. 21–28.

[35] E. Mariconti, L. Onwuzurike, P. Andriotis, E. D. Cristofaro, G. J. Ross, and G. Stringhini, "MaMaDroid: Detecting Android malware by building Markov chains of behavioral models," *ACM Trans. Privacy Secur.*, vol. 22, no. 2, pp. 1–34, 2017.

[36] V. Seixas and K. Elish, "High-accuracy Android ransomware detection using SMALI code-based analysis," in *Proc. Int. Conf. Artif. Intell., Comput., Data Sci. Appl. (ACDSA)*, Aug. 2025, pp. 1–6.

[37] M. Sun, T. Wei, and J. C. S. Lui, "TaintART: A practical multi-level information-flow tracking system for Android RunTime," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 331–342.

[38] D. Carlin, A. Cowan, P. O'Kane, and S. Sezer, "The effects of traditional anti-virus labels on malware detection using dynamic runtime opcodes," *IEEE Access*, vol. 5, pp. 17742–17752, 2017.

[39] M. Harman and P. O'Hearn, "From start-ups to scale-ups: Opportunities and open problems for static and dynamic program analysis," in *Proc. IEEE 18th Int. Work. Conf. Source Code Anal. Manipulation (SCAM)*, Sep. 2018, pp. 1–23.

[40] P. Feng, J. Ma, C. Sun, X. Xu, and Y. Ma, "A novel dynamic Android malware detection system with ensemble learning," *IEEE Access*, vol. 6, pp. 30996–31011, 2018.

[41] J. Jeon, J. H. Park, and Y.-S. Jeong, "Dynamic analysis for IoT malware detection with convolution neural network model," *IEEE Access*, vol. 8, pp. 96899–96911, 2020.

[42] G. Sathyadevi, J. Abishek, and B. S. Shakthieiswaran, "DynaShield: Android malware detection using dynamic analysis of network traffic," in *Proc. Int. Conf. Syst., Comput., Autom. Netw. (ICSCAN)*, Dec. 2024, pp. 1–6.

[43] H. Long, Z. Tian, and Y. Liu, "Detecting Android malware based on dynamic feature sequence and attention mechanism," in *Proc. IEEE 5th Int. Conf. Cryptography, Secur. Privacy (CSP)*, Jan. 2021, pp. 129–133.

[44] T. Palit, J. F. Moon, F. Monrose, and M. Polychronakis, "DynPTA: Combining static and dynamic analysis for practical selective data protection," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2021, pp. 1919–1937.

[45] N. Dalhy and K. Elish, "Stealthy no more: An effective ensemble learning approach for detecting Android RATs," in *Proc. IEEE/ACIS 23rd Int. Conf. Softw. Eng. Res., Manage. Appl. (SERA)*, May 2025, pp. 194–200.

[46] Saurabh, "Advance malware analysis using static and dynamic methodology," in *Proc. Int. Conf. Adv. Comput. Telecommun. (ICACAT)*, Dec. 2018, pp. 1–5.

[47] Y. Feng, S. Anand, I. Dillig, and A. Aiken, "Apposcopy: Semantics-based detection of Android malware through static analysis," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, Nov. 2014, pp. 576–587.

[48] S. Tikhomirov, E. Voskresenskaya, I. Ivanitskiy, R. Takhaviev, E. Marchenko, and Y. Alexandrov, "SmartCheck: Static analysis of Ethereum smart contracts," in *Proc. IEEE/ACM 1st Int. Workshop Emerg. Trends Softw. Eng. Blockchain (WETSEB)*, May 2018, pp. 9–16.

[49] *Malwarebazaar: Malware Sample Exchange Platform*. Accessed: May 2025. [Online]. Available: https://bazaar.abuse.ch/

[50] D. Arp, M. Spreitzenbarth, M. Hübner, H. Gascon, and K. Rieck, "Drebin: Effective and explainable detection of Android malware in your pocket," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2014, pp. 23–26.

[51] K. Allix, T. F. Bissyandé, J. Klein, and Y. Le Traon, "AndroZoo: Collecting millions of Android apps for the research community," in *Proc. 13th Int. Conf. Mining Softw. Repositories*, May 2016, pp. 468–471.

[52] D. Gibert, C. Mateu, J. Planes, and R. Vicens, "Classification of malware by using structural entropy on convolutional neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 7759–7764.

[53] D. Yuxin and Z. Siyi, "Malware detection based on deep learning algorithm," *Neural Comput. Appl.*, vol. 31, no. 2, pp. 461–472, Feb. 2019.
[54] B. J. Walton, M. E. Khatun, J. M. Ghawaly, and A. Ali-Gombe, "Exploring large language models for semantic analysis and categorization of Android malware," in *Proc. Annu. Comput. Secur. Appl. Conf. Workshops (ACSAC Workshops)*, Dec. 2024, pp. 248–254.
[55] A. Arora, S. K. Peddoju, V. Chouhan, and A. Chaudhary, "Hybrid Android malware detection by combining supervised and unsupervised learning," in *Proc. 24th Annu. Int. Conf. Mobile Comput. Netw.*, Oct. 2018, pp. 798–800.
[56] *PRISMA Statement*. Accessed: Mar. 2025. [Online]. Available: https://www.prisma-statement.org/
[57] B. Lee and C. Easttom, "A novel framework for malware analysis: Integrating large language models with heuristic techniques," in *Proc. IEEE 15th Annu. Comput. Commun. Workshop Conf. (CCWC)*, Jan. 2025, pp. 516–524.
[58] M. Dong, L. Liu, Q. Guo, H. Bai, R. Gong, Y. Bai, W. He, Z. Wang, G. Xu, and J. Zhang, "LeoDroid: An LLM-based few-shot multi-label detection for Android malware," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2025, pp. 294–306.
[59] Y. He, H. She, X. Qian, X. Zheng, Z. Chen, Z. Qin, and L. Cavallaro, "On benchmarking code LLMs for Android malware analysis," in *Proc. 34th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jun. 2025, pp. 153–160.
[60] B. Souani, A. Khanfir, A. Bartel, K. Allix, and Y. L. Traon, "Android malware detection using BERT," in *Proc. Int. Conf. Appl. Cryptography Netw. Secur.*, 2022, pp. 575–591.
[61] X. Zhang, H. Meng, Q. Li, Y. Tan, and L. Zhang, "Large language models powered malicious traffic detection: Architecture, opportunities, and case study," *IEEE Netw.*, vol. 39, no. 5, pp. 51–57, Sep. 2025.
[62] S. Park, D. Min, T. A. Nguyen, and E. Choi, "Metaverse security: LLM-based malicious code detection through token optimization," in *Proc. Int. Conf. Metaverse Comput., Netw. Appl. (MetaCom)*, Aug. 2025, pp. 316–323.
[63] A. Charoenthanakitkul, P. Viboonsang, and S. Kosolsombat, "Optimizing malware detection with random forest, XGBoost, LightGBM, and LLM-reporting," in *Proc. IEEE Int. Conf. Cybern. Innov. (ICCI)*, Apr. 2025, pp. 1–5.
[64] J.-H. Lang and T. Schreck, "Leveraging LLMs for memory forensics: A comparative analysis of malware detection," *Digit. Threats, Res. Pract.*, vol. 6, no. 4, pp. 1–20, Dec. 2025.
[65] N. M. Kumar, F. T. Lisa, and S. R. Islam, "Prompt chaining-assisted malware detection: A hybrid approach utilizing fine-tuned LLMs and domain knowledge-enriched cybersecurity knowledge graphs," in *Proc. IEEE Int. Conf. Big Data (BigData)*, Dec. 2024, pp. 1672–1677.
[66] S. Altamimi and M. Ababneh, "Detecting spam and malware using BERT and LLMs," in *Proc. 25th Int. Arab Conf. Inf. Technol. (ACIT)*, Dec. 2024, pp. 1–6.
[67] A. A. Hossain, M. K. PK, J. Zhang, and F. Amsaad, "Malicious code detection using LLM," in *Proc. NAECON IEEE Nat. Aerosp. Electron. Conf.*, Jul. 2024, pp. 414–416.
[68] H. S. Anderson and P. Roth, "EMBER: An open dataset for training static PE malware machine learning models," 2018, *arXiv:1804.04637*.
[69] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.
[70] M. Al-Hawawreh, E. Sitnikova, and N. Aboutorab, "X-IIoTID: A connectivity-agnostic and device-agnostic intrusion data set for industrial Internet of Things," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3962–3977, Mar. 2022.
[71] N. Moustafa, "A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets," *Sustain. Cities Soc.*, vol. 72, Sep. 2021, Art. no. 102994.
[72] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, Jun. 2023.
[73] S. S. Gu, Y. Iwasawa, T. Kojima, Y. Matsuo, and M. Reid, "Large language models are zero-shot reasoners," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 22199–22213.
[74] N. Cancedda, R. Dessi, J. Dwivedi-Yu, E. Hambro, M. Lomeli, R. Raileanu, T. Schick, T. Scialom, and L. Zettlemoyer, "Toolformer: Language models can teach themselves to use tools," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 68539–68551.
[75] *World's Most Dangerous Malware Emotet Disrupted Through Global Action*. Accessed: Jun. 2025. [Online]. Available: https://www.europol.europa.eu/media-press/newsroom/news/world%E2%80%99s-most-dangerous-malware-emotet-disrupted-through-global-action
[76] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116.
[77] D. S. Keyes, B. Li, G. Kaur, A. H. Lashkari, F. Gagnon, and F. Massicotte, "EntropLyzer: Android malware classification and characterization using entropy analysis of dynamic characteristics," in *Proc. Reconciling Data Anal., Autom., Privacy, Secur., Big Data Challenge (RDAAPS)*, May 2021, pp. 1–12.
[78] L. Li, T. F. Bissyandé, and J. Klein, "Rebooting research on detecting repackaged Android apps: Literature review and benchmark," *IEEE Trans. Softw. Eng.*, vol. 47, no. 4, pp. 676–693, Apr. 2021.
[79] R. S. de Zaldivar, A. H. Celdrán, J. von der Assen, P. M. S. Sánchez, G. Bovet, G. M. Pérez, and B. Stiller, "MalwSpecSys: A dataset containing syscalls of an IoT spectrum sensor affected by heterogeneous malware," IEEE Dataport, 2022, doi: 10.21227/nvmb-eg69.
[80] B. Gençaydin, C. N. Kahya, F. Demirkiran, B. Düzgün, A. Çayir, and H. Dag, "Benchmark static API call datasets for malware family classification," in *Proc. 7th Int. Conf. Comput. Sci. Eng. (UBMK)*, Sep. 2022, pp. 1–5.
[81] *General Data Protection Regulation (GDPR)*. Accessed: Jul. 2025. [Online]. Available: https://gdpr-info.eu/
[82] *NIST AI Risk Management Framework*. Accessed: Jul. 2025. [Online]. Available: https://www.nist.gov/itl/ai-risk-management-framework



JOYCE CHAMPIE received the B.Sc. degree in software engineering from Coventry University, Ghana. She is currently pursuing the M.Sc. degree in computer science with Florida Polytechnic University, Lakeland, FL, USA. She also works as a Graduate Assistant. Her academic and project experience includes work in cybersecurity, mobile security, software security, and analytics. She has contributed to research on Android malware detection, code-smell datasets, and large-language-model-based threat analysis. She has also worked on industry-aligned projects involving data analytics, SQL-based pipelines, and ETL workflow development. She has been recognized with awards for academic and research excellence, including a first-place poster presentation award. She is a Student Member of the ACM.



KARIM ELISH (Senior Member, IEEE) received the M.Sc. and Ph.D. degrees in computer science from Virginia Tech, in 2011 and 2015, respectively. He is currently an Associate Professor in computer science with Florida Polytechnic University, USA. His research interests include mobile security and privacy, software security and analytics, applied AI/ML, and large language model–driven software assurance. He has published numerous peer-reviewed publications in leading venues. He has served as a reviewer and a program committee member for various IEEE and ACM journals and conferences. He is a Senior Member of ACM.